C++ unique_lock,共享锁和递归锁

Posted on 2023-09-02 11:27:18 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Up8cDw1Dl6otzUxbSdQksUjRua)

简介

本文介绍C++ 并发中使用的其他类型的锁，包括 `unique_lock` ， `shared_lock` ，以及 `recursive_lock` 等。 `shared_lock` 和 `unique_lock` 比较常用，而 `recursive_lock` 用的不多，或尽可能规避用这种锁。

unique_lock

`unique_lock` 和 `lock_guard` 基本用法相同，构造时默认加锁，析构时默认解锁，但 `unique_lock` 有个好处就是可以手动解锁。这一点尤为重要，方便我们控制锁住区域的粒度(加锁的范围大小),也能支持和条件变量配套使用，至于条件变量我们之后再介绍，本文主要介绍锁的相关操作。

```
1. //unique_lock 基本用法
2. std::mutex mtx;
3. int shared_data = 0;
4. void use_unique() {
5.     //lock可自动解锁，也可手动解锁
6.     std::unique_lock<std::mutex> lock(mtx);
7.     std::cout << "lock success" << std::endl;
8.     shared_data++;
9.     lock.unlock();
10. }
```

我们可以通过 `unique_lock` 的 `owns_lock` 判断是否持有锁

```
1. //可判断是否占有锁
2. void owns_lock() {
3.     //lock可自动解锁，也可手动解锁
4.     std::unique_lock<std::mutex> lock(mtx);
5.     shared_data++;
6.     if (lock.owns_lock()) {
7.         std::cout << "owns lock" << std::endl;
8.     }
9.     else {
10.        std::cout << "doesn't own lock" << std::endl;
11.    }
12.
13.    lock.unlock();
14.    if (lock.owns_lock()) {
15.        std::cout << "owns lock" << std::endl;
16.    }
17.    else {
18.        std::cout << "doesn't own lock" << std::endl;
19.    }
20. }
```

上述代码输出

```
1. owns lock
2. doesn't own lock
```

`unique_lock` 可以延迟加锁

```
1.  //可以延迟加锁
2. void defer_lock() {
3.     //延迟加锁
4.     std::unique_lock<std::mutex> lock(mtx, std::defer_lock);
5.     //可以加锁
6.     lock.lock();
7.     //可以自动析构解锁，也可以手动解锁
8.     lock.unlock();
9. }
```

那我们写一段代码综合运用 `owns_lock` 和 `defer_lock`

```
1. //同时使用owns和defer
2. void use_own_defer() {
3.     std::unique_lock<std::mutex>  lock(mtx);
4.     // 判断是否拥有锁
5.     if (lock.owns_lock())
6.     {
7.         std::cout << "Main thread has the lock." << std::endl;
8.     }
9.     else
10.    {
11.        std::cout << "Main thread does not have the lock." << std::endl;
12.    }
13.
14.    std::thread t([]() {
15.        std::unique_lock<std::mutex> lock(mtx, std::defer_lock);
16.
17.        // 判断是否拥有锁
18.        if (lock.owns_lock())
19.        {
20.            std::cout << "Thread has the lock." << std::endl;
21.        }
22.        else
23.        {
24.            std::cout << "Thread does not have the lock." << std::endl;
25.        }
26.
27.        // 加锁
28.        lock.lock();
29.
30.        // 判断是否拥有锁
31.        if (lock.owns_lock())
32.        {
33.            std::cout << "Thread has the lock." << std::endl;
34.        }
35.        else
36.        {
37.            std::cout << "Thread does not have the lock." << std::endl;
38.        }
39.
40.        // 解锁
41.        lock.unlock();
42.        });
```

```
    t.join();
}
```

上述代码回依次输出, 但是程序会阻塞，因为子线程会卡在加锁的逻辑上，因为主线程未释放锁，而主线程又等待子线程退出，导致整个程序卡住。

```
Main thread has the lock.
Thread does not have the lock.
```

和 `lock_guard` 一样， `unique_lock` 也支持领养锁

```
//同样支持领养操作
void use_own_adopt() {
    mtx.lock();
    std::unique_lock<std::mutex> lock(mtx, std::adopt_lock);
    if (lock.owns_lock()) {
        std::cout << "owns lock" << std::endl;
    }
    else {
        std::cout << "does not have the lock" << std::endl;
    }
    lock.unlock();
}
```

尽管是领养的，但是打印还是会出现 `owns lock` ，因为不管如何锁被加上，就会输出 `owns lock` 。

既然 `unique_lock` 支持领养操作也支持延迟加锁，那么可以用两种方式实现前文 `lock_guard` 实现的 `swap` 操作。

```
1. //之前的交换代码可以可以用如下方式等价实现
2. int a = 10;
3. int b = 99;
4. std::mutex  mtx1;
5. std::mutex  mtx2;
6.
7. void safe_swap() {
8.     std::lock(mtx1, mtx2);
9.     std::unique_lock<std::mutex> lock1(mtx1, std::adopt_lock);
10.    std::unique_lock<std::mutex> lock2(mtx2, std::adopt_lock);
11.    std::swap(a, b);
12.    //错误用法
13.    //mtx1.unlock();
14.    //mtx2.unlock();
15. }
16.
17. void safe_swap2() {
18.
19.    std::unique_lock<std::mutex> lock1(mtx1, std::defer_lock);
20.    std::unique_lock<std::mutex> lock2(mtx2, std::defer_lock);
21.    //需用lock1,lock2加锁
22.    std::lock(lock1, lock2);
23.    //错误用法
24.    //std::lock(mtx1, mtx2);
25.    std::swap(a, b);
26. }
```

大家注意一旦 `mutex` 被 `unique_lock` 管理，加锁和释放的操作就交给 `unique_lock` ，不能调用 `mutex` 加锁和解锁，因为锁的使用权已经交给 `unique_lock` 了。

我们知道 `mutex` 是不支持移动和拷贝的，但是 `unique_lock` 支持移动，当一个 `mutex` 被转移给 `unique_lock` 后，可以通过unique_ptr转移其归属权.

```
1. //转移互斥量所有权
2. //互斥量本身不支持move操作，但是unique_lock支持
3. std::unique_lock <std::mutex>  get_lock() {
4.     std::unique_lock<std::mutex>  lock(mtx);
5.     shared_data++;
6.     return lock;
7. }
8.
9. void use_return() {
10.     std::unique_lock<std::mutex> lock(get_lock());
11.     shared_data++;
12. }
```

锁的粒度表示加锁的精细程度，一个锁的粒度要足够大，保证可以锁住要访问的共享数据。

同时一个锁的粒度要足够小，保证非共享数据不被锁住影响性能。

而 `unique_ptr` 则很好的支持手动解锁。

```
1. void precision_lock() {
2.     std::unique_lock<std::mutex> lock(mtx);
3.     shared_data++;
4.     lock.unlock();
5.     //不设计共享数据的耗时操作不要放在锁内执行
6.     std::this_thread::sleep_for(std::chrono::seconds(1));
7.     lock.lock();
8.     shared_data++;
9. }
```

共享锁

试想这样一个场景，对于一个DNS服务，我们可以根据域名查询服务对应的ip地址，它很久才更新一次，比如新增记录，删除记录或者更新记录等。平时大部分时间都是提供给外部查询，对于查询操作，即使多个线程并发查询不加锁也不会有问题，但是当有线程修改DNS服务的ip记录或者增减记录时，其他线程不能查询，需等待修改完再查询。或者等待查询完，线程才能修改。也就是说读操作并不是互斥的，同一时间可以有多个线程同时读，但是写和读是互斥的，写与写是互斥的，简而言之，写操作需要独占锁。而读操作需要共享锁。

要想使用共享锁，需使用共享互斥量 `std::shared_mutex` ，`std::shared_mutex` 是C++17标准提

出的。
C++14标准可以使用 `std::shared_time_mutex` ,

`std::shared_mutex` 和 `std::shared_timed_mutex` 都是用于实现多线程并发访问共享数据的互斥锁，但它们之间存在一些区别：

1. `std::shared_mutex` :

1. * 提供了 `lock()`, `try_lock()`, 和 `try_lock_for()` 以及 `try_lock_until()` 函数，这些函数都可以用于获取互斥锁。
2. * 提供了 `try_lock_shared()` 和 `lock_shared()` 函数，这些函数可以用于获取共享锁。
3. * 当 `std::shared_mutex` 被锁定后，其他尝试获取该锁的线程将会被阻塞，直到该锁被解锁。

1. `std::shared_timed_mutex` :

1. * 与 `std::shared_mutex` 类似，也提供了 `lock()`, `try_lock()`, 和 `try_lock_for()` 以及 `try_lock_until()` 函数用于获取互斥锁。
2. * 与 `std::shared_mutex` 不同的是，它还提供了 `try_lock_shared()` 和 `lock_shared()` 函数用于获取共享锁，这些函数在尝试获取共享锁时具有超时机制。
3. * 当 `std::shared_timed_mutex` 被锁定后，其他尝试获取该锁的线程将会被阻塞，直到该锁被解锁，这与 `std::shared_mutex` 相同。然而，当尝试获取共享锁时，如果不能立即获得锁，`std::shared_timed_mutex` 会设置一个超时，超时过后如果仍然没有获取到锁，则操作将返回失败。

因此， `std::shared_timed_mutex` 提供了额外的超时机制，这使得它在某些情况下更适合于需要处理超时的并发控制。然而，如果不需要超时机制，可以使用更简单的 `std::shared_mutex` 。

C++11标准没有共享互斥量，可以使用boost提供的 `boost::shared_mutex` 。

如果我们想构造共享锁，可以使用 `std::shared_lock` ，如果我们想构造独占锁, 可以使用 `std::lock_gurad` .

我们用一个类 `DNService` 代表DNS服务，查询操作使用共享锁，而写操作使用独占锁，可以是如下方式的。

```
1. class DNService {
2. public:
3.     DNService() {}
4.     //读操作采用共享锁
5.     std::string QueryDNS(std::string dnsname) {
6.         std::shared_lock<std::shared_mutex> shared_locks(_shared_mtx);
7.         auto iter = _dns_info.find(dnsname);
8.         if (iter != _dns_info.end()) {
9.             return iter->second;
10.        }
11.
12.        return "";
13.    }
14.
15.    //写操作采用独占锁
16.    void AddDNSInfo(std::string dnsname, std::string dnsentry) {
17.        std::lock_guard<std::shared_mutex>  guard_locks(_shared_mtx);
18.        _dns_info.insert(std::make_pair(dnsname, dnsentry));
19.    }
20. private:
21.    std::map<std::string, std::string> _dns_info;
22.    mutable std::shared_mutex  _shared_mtx;
23. };
```

`QueryDNS` 用来查询dns信息，多个线程可同时访问。
`AddDNSInfo` 用来添加dns信息，属独占锁，同一时刻只有一个线程在修改。

递归锁

有时候我们在实现接口的时候内部加锁，接口内部调用完结束自动解锁。会出现一个接口调用另一个接口的情况，如果用普通的 `std::mutex` 就会出现卡死，因为嵌套加锁导致卡死。但是我们可以使用递归锁。

但我个人并不推荐递归锁，可以从设计源头规避嵌套加锁的情况，我们可以将接口相同的功能抽象出来，统一加锁。下面的设计演示了如何使用递归锁

```
class RecursiveDemo {
public:
    RecursiveDemo() {}
    bool QueryStudent(std::string name) {
        std::lock_guard<std::recursive_mutex>  recursive_lock(_recursive_mtx);
        auto iter_find = _students_info.find(name);
        if (iter_find == _students_info.end()) {
            return false;
        }

        return true;
    }

    void AddScore(std::string name, int score) {
        std::lock_guard<std::recursive_mutex>  recursive_lock(_recursive_mtx);
        if (!QueryStudent(name)) {
            _students_info.insert(std::make_pair(name, score));
            return;
        }

        _students_info[name] = _students_info[name] + score;
    }

    //不推荐采用递归锁，使用递归锁说明设计思路并不理想，需优化设计
    //推荐拆分逻辑，将共有逻辑拆分为统一接口
    void AddScoreAtomic(std::string name, int score) {
        std::lock_guard<std::recursive_mutex>  recursive_lock(_recursive_mtx);
        auto iter_find = _students_info.find(name);
        if (iter_find == _students_info.end()) {
            _students_info.insert(std::make_pair(name, score));
            return;
        }

        _students_info[name] = _students_info[name] + score;
        return;
    }
private:

    std::map<std::string, int> _students_info;
    std::recursive_mutex   _recursive_mtx;
};
```

我们可以看到 `AddScore` 函数内部调用了 `QueryStudent` ，所以采用了递归锁。

但是我们同样可以改变设计，将两者公有的部分抽离出来生成一个新的接口 `AddScoreAtomic` .

`AddScoreAtomic` 可以不适用递归锁，照样能完成线程安全操作的目的。

总结

本文介绍了unique_lock，共享锁，递归锁等的使用，较为全面的介绍了这几种锁的使用场景和潜在风险。

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

源码链接

https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(3072) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(560) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)